

Testing Apollo

TESTING APOLLO INTRODUCTION



Marques Woodson

STAFF SOFTWARE ENGINEER

@mwq27 www.marqueswoodson.com



Overview



Full-Stack testing of an Apollo app

Apollo Client with React

Apollo Server on NodeJS

Testing queries and mutations

- Using React Testing Library

Resolvers and Mutations on the server

- MockLists and data mocking
- Introspection
- Integration tests



Apollo Client



Prerequisites to get the most out of this course

React Hooks

Understand how hooks like `useEffect` and `useState` work

Apollo and GraphQL

Basic knowledge of Apollo and GraphQL

NodeJS

Basic NodeJS knowledge



**Declarative library for
GraphQL**

**Officially supported
by React**

**Community support
for Vue, Svelte,
Angular, Ember**

Apollo Client



Apollo Client Hooks of Interest

useQuery

for fetching data

useMutation

for editing data





useQuery Hook

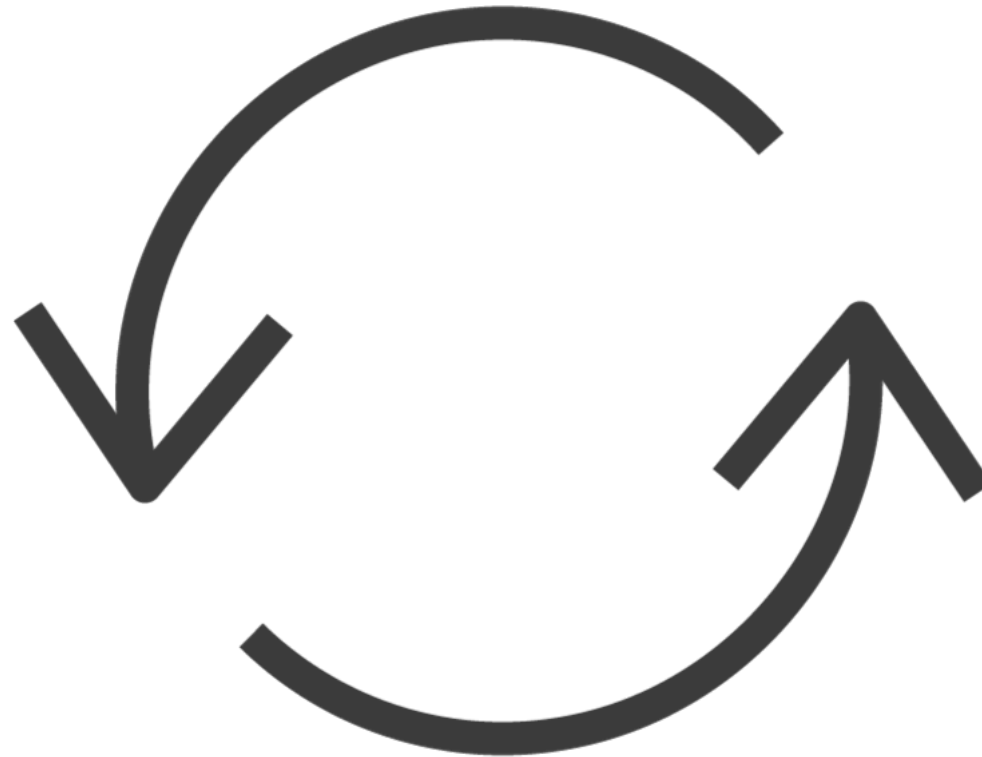
Works with useEffect and other React hooks to make requests to a GraphQL backend.



GraphQL Database



Client Code



useQuery Return Values



loading

Boolean letting you know that work is still happening



error

Error object with network and Apollo related errors



data

Data returned from the GraphQL database



Error Policies

Apollo error policies can be set on every request



None

Set by default. All errors handled like network errors.



Ignore

Error is ignored on the client side.



All

Recommended policy. Handle GraphQL as well as network errors.



addProduct.jsx

useMutation

```
const [addProduct, { loading, error }] = useMutation(ADD_PRODUCT_QUERY);
```

React Context

Share functionality with multiple components

index.js

```
const client = new ApolloClient({  
  cache: new InMemoryCache(),  
  link: new HttpLink({  
    uri: 'https://path-to-graphql-server.com/',  
  }),  
});
```

```
<ApolloProvider client={client}>  
  <App />  
</ApolloProvider>
```

Mocked Provider

Apollo context provider used for mocking in client tests

App.test.js

```
<MockedProvider mocks={mocks} addTypename={false}>  
  <App />  
</MockedProvider>
```

```
const mocks = [
  {
    request: {
      query: GET_PRODUCTS_QUERY,
      variables: { id: 1 }
    },
    result: {
      data: {
        product: { id: '1', name:
'Life Jacket', price: '$39.99' },
      },
    },
  },
];
```

- ◀ Request and response mock for query to get products
- ◀ Query
- ◀ Results to return



Apollo Server





Apollo Server

- Query and mutation resolvers
- Introspection
- Integration tests



Schema

Defines the shape and structure of your data. It also describes what queries and mutations are available to the client.



Resolvers

Responsible for populating data for every field in the schema

resolvers.js

```
const resolvers = {  
  Query: {  
    products: (parent, args, ctx) => {...}  
  },  
  Mutation: {  
    addProduct: (parent, args, ctx) => {...}  
  },  
  Product : {  
    name: (parent, args, ctx) => {...}  
  }  
};
```

Save time by using Apollo's
default resolvers.



Mocks



With Mocks Set to True

Query

```
query {  
  product {  
    name  
    price  
    id  
  }  
}
```

Results

```
data {  
  product {  
    name: "Hello World",  
    price: "2",  
    id: 4  
  }  
}
```

Limits of “mocks: true”

User defined resolvers are ignored

Inability to customize mocked values



MockList

Automatically generates a list of data based on your schema definition



```
const mocks = {  
  Query: () => ({  
    products: () => new MockList(5),  
    users: () => new MockList([5, 15]),  
  })  
};
```

- ◀ Return 5 mocked *products*
- ◀ Return somewhere between 5 and 15 mocked *users*




```
const server = new ApolloServer({  
  typeDefs,  
  resolvers,  
  mocks,  
  mockEntireSchema: false  
});
```

- ◀ Keep existing, user-defined resolvers from being mocked.





Schema not written in JS?

Schema's can be written in multiple languages.
You can use introspection to get schema
information to be used for mocking.

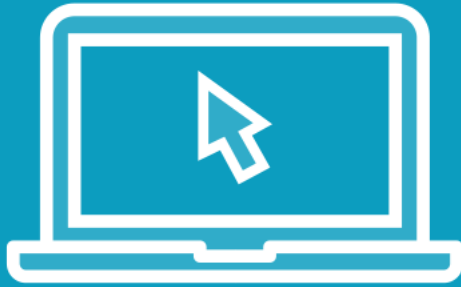


Introspection

A method used to ask a GraphQL schema for information about itself, like what queries it supports.



Demo



Apollo client features, including
MockedProvider, useQuery and
useMutation hooks, and error handling

Apollo server features, including
resolvers, introspection, and integration
tests





React App

Apollo Client v3

- @apollo/client package now holds all client-side modules



Features we'll
be testing on
the client

Dynamic content in the Stories section

- `useQuery` and `<Query />`

Ability to update a story

- `useMutation`



Connecting
with the
server

Apollo Server link

- <https://codesandbox.io/s/apollo-server-container-nws8z>

